FATEC POMPEIA

Introdução à Computação em Nuvem · ISW055 · 161_SIST. INTELIGENTES_N

# P2

## Relatório consolidado

arquitetura e atividades do 2º bimestre

Avaliação individual · 2026.2

Gabriel Roberto de Assis

Produto: **UniRide — Carona Universitária Compartilhada**
https://github.com/Gabriel-Assis-22/Atividade_03_Claude

Prof. Allan Lincoln Rodrigues Siriani
Pompeia, 09/11/2026

# Sumário

1 Introdução ..... 2
2 Metodologia ..... 2
3 Arquitetura do SaaS ..... 4
3.1 Visão geral e componentes ..... 4
3.2 Onde roda ..... 5
3.3 Fluxo de uma requisição ..... 6
3.4 Fluxo de assinatura ..... 7
3.5 Modelo de dados do domínio ..... 8
3.6 Segurança ..... 9
3.7 Observabilidade, CI/CD e documentação da API ..... 10
3.8 Camadas IaaS, PaaS e SaaS do projeto ..... 11
4 Quadro de entregas ..... 13
5 Atividades realizadas ..... 14
5.1 Atividade 7 — Plano Premium — cobrança de verdade com Stripe ..... 14
5.2 Atividade 8 — SaaS, PaaS, IaaS — e a documentação que o projeto nunca teve ..... 17
5.3 Atividade 9 — Fork e plano de negócio ..... 20
5.4 Atividade 10 — Seu negócio no ar — o domínio no lugar dos filmes ..... 23
5.5 Atividade 11 — SaaS completo — planos, limites e assinatura com Stripe ..... 28
6 Considerações finais ..... 32
7 Declaração de autoria ..... 32

# 1 Introdução

A disciplina **Introdução à Computação em Nuvem (ISW055)**, ministrada na Fatec Pompeia, adota uma abordagem prática e progressiva para o ensino de sistemas distribuídos: em vez de tratar conceitos de infraestrutura, plataformas e software como serviço de forma puramente teórica, o curso conduz a construção real de uma arquitetura de microsserviços em nuvem ao longo de todo o semestre. Enquanto o primeiro bimestre concentrou-se em desacoplar serviços fundamentais — autenticação independente com JWT, controle de acesso baseado em papéis (RBAC), auditoria centralizada em Redis Streams e armazenamento de objetos S3 — utilizando um catálogo cinematográfico temático como laboratório técnico, o segundo bimestre propôs converter essa fundação tecnológica em um produto comercial no modelo **Software as a Service** (SaaS).

A partir dessa evolução nasceu o **UniRide**, uma plataforma SaaS de caronas universitárias compartilhadas voltada a estudantes e condutores acadêmicos que se deslocam diariamente entre cidades da região centro-oeste paulista (como Marília, Bauru, Garça e Pompeia). O produto ataca uma dor concreta enfrentada no cotidiano do campus: alunos sem transporte próprio chegam a desembolsar mais de R$ 400,00 mensais em passagens intermunicipais e perdem de duas a três horas por noite aguardando itinerários rígidos, ao passo que dezenas de colegas condutores percorrem exatamente as mesmas rodovias com assentos vazios e alto custo individual de combustível. Até então, essa demanda dependia de grupos informais de mensagens instantâneas sem confirmação de assento, histórico ou verificação mínima de identidade.

Reaproveitando integralmente a malha de containers Docker desenvolvida na etapa anterior, o UniRide substituiu o domínio de filmes por fluxos completos de mobilidade acadêmica — cadastro de veículos com fotos em Object Storage, oferta e busca parametrizada de rotas, reserva atômica de vagas com proteção contra adulteração entre usuários e monetização recorrente via Stripe (**Plano Passageiro Free** gratuito limitado a duas reservas ativas simultâneas versus **Plano UniRide Pro** por R$ 14,90/mês com reservas ilimitadas e autoatendimento pelo portal do cliente).

Este relatório consolidado cumpre dupla finalidade na avaliação bimestral (P2): na Seção 3, documenta em profundidade a arquitetura efetivamente construída e em operação do UniRide SaaS, servindo como manual técnico de manutenção e governança do sistema; nas Seções 4 e 5, apresenta o quadro cronológico e as fichas individuais auditáveis das Atividades 7 a 11, confrontando prazos planejados, horários exatos de entrega extraídos do controle de versão e evidências verificáveis.

# 2 Metodologia

O desenvolvimento das entregas do segundo bimestre seguiu um fluxo iterativo apoiado em conteinerização com Docker Compose, separação de camadas em **Domain-Driven Design** (DDD) nos microsserviços `.NET 10` (`Domain`, `Application`, `Infrastructure` e `Api`), interface reativa em `Angular 19` servida por proxy reverso `Nginx`, persistência relacional em `MariaDB` com migrações versionadas via `FluentMigrator` e consultas parametrizadas com `Dapper`, armazenamento de objetos no `Garage S3` via assinatura criptográfica AWS SigV4 e mensageria de auditoria em `Redis 7 Streams`. Para o ciclo financeiro de assinaturas, utilizou-se o SDK oficial `Stripe.net` operando em modo de teste (**Test Mode**), garantindo conformidade com o padrão PCI-DSS sem tráfego direto de dados sensíveis de cartão pelo backend.

No tocante à organização de versionamento no GitHub, o trabalho distribuiu-se em dois repositórios públicos complementares sob a conta do autor. As Atividades 7 (integração inicial do Plano Premium com Stripe sobre o catálogo) e 8 (classificação em camadas IaaS/PaaS/SaaS, engenharia reversa em `DOCUMENTACAO.md` e especificação OpenAPI/Swagger) foram implementadas e commitadas diretamente no repositório original

do primeiro bimestre (github.com/Gabriel-Assis-22/Atividade_02_Claude), preservando intacta toda a árvore histórica de commits desde agosto de 2026. Na Atividade 9, iniciou-se o repositório dedicado ao novo produto (github.com/Gabriel-Assis-22/Atividade_03_Claude) a partir da base consolidada do projeto anterior, mantendo ambos os repositórios abertos e cruzados por links diretos nos respectivos arquivos `README.md` para assegurar total transparência e rastreabilidade dos hashes originais.

Todas as datas e horários realizados declarados neste documento foram extraídos sem edição manual diretamente dos metadados oficiais do Git e da API pública do GitHub, convertidos para o fuso horário local de Brasília (`America/Sao_Paulo`, UTC-3). Nos repositórios locais, utilizou-se o comando padronizado `git log --date=format:"%d/%m/%Y %H:%M" --format="%h %ad %s"`, confrontando cada hash com a respectiva página web do commit no GitHub e, no caso do marco de abertura do novo repositório na Atividade 9, com o registro temporal de criação da Issue #1 mencionando `@siriani`. Adicionalmente, os testes funcionais de validação das regras de negócio das Atividades 10 e 11 (bloqueios RBAC/Anti-IDOR `HTTP 403`, limite de reservas do plano gratuito, métricas de MRR e trilha de eventos no Redis) foram executados contra a stack completa em execução no Docker Compose.

# 3 Arquitetura do SaaS

Esta seção descreve a arquitetura real do **UniRide SaaS** tal como implementada nos repositórios e orquestrada em containers Docker, detalhando cada microsserviço, a infraestrutura de execução, os fluxos síncronos e assíncronos de negócio e cobrança, o modelo relacional e os mecanismos de segurança.

## 3.1 Visão geral e componentes

O sistema é modularizado em microsserviços independentes interligados pela rede virtual privada `app-network` (driver `bridge`). Apenas o container `frontend` (Nginx) possui porta publicada para o mundo externo (`:8208`), atuando simultaneamente como servidor da Single Page Application (SPA) e **API Gateway / Reverse Proxy** para o `backend` e para o armazenamento de imagens. Todos os demais serviços operam estritamente isolados na rede interna com a diretiva `expose`.

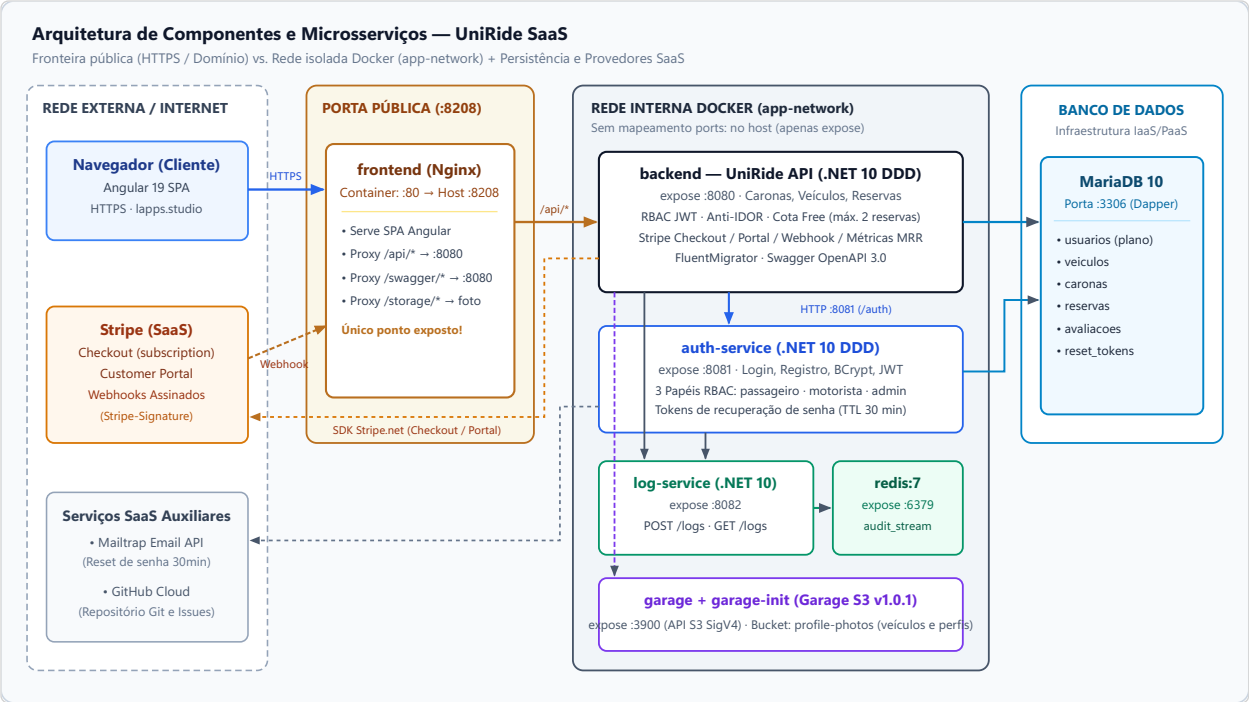



Figura 1: Diagrama de componentes, fronteira pública e rede interna Docker do UniRide SaaS

| Componente | Responsabilidade | Tecnologia / imagem | Público? |
|---|---|---|---|
| `frontend` | Entrega da SPA (Landing Page, Feed de Caronas, Painel do Condutor/Passageiro/Admin) e proxy reverso para `/api/`, `/swagger/` e `/storage/` | Angular 19 + `nginx:alpine` (`./frontend`) | Sim (`:8208` / domínio) |
| `backend` | API central do negócio: CRUD de veículos e caronas, reserva atômica de vagas, enforcement de cotas SaaS, proxy de mídia S3 e integração Stripe | ASP.NET Core 10 C# DDD (`./backend`) | Não (`expose :8080`) |
| `auth-service` | Gestão de identidade desacoplada: registro, hash BCrypt, login com emissão de JWT, 3 | ASP.NET Core 10 C# DDD (`./auth-service`) | Não (`expose :8081`) |

| Componente | Responsabilidade | Tecnologia / imagem | Público? |
|---|---|---|---|
| | papéis RBAC e recuperação de senha com TTL de 30 min | | |
| `log-service` | Centralização de auditoria: recepção assíncrona de eventos (`POST /logs`) e consulta cronológica protegida para administradores (`GET /logs`) | ASP.NET Core 10 Web API (`./log-service`) | Não (`expose :8082`) |
| `redis` | Persistência **append-only** ordenada no tempo da trilha de auditoria e eventos de negócio na estrutura `audit_stream` (`XADD` / `XREVRANGE`) | `redis:7-alpine` | Não (`expose :6379`) |
| `MariaDB` | Persistência relacional ACID de usuários, planos/assinaturas, veículos, caronas, reservas, avaliações e tokens de redefinição de senha | MariaDB 10 + Dapper + FluentMigrator | Gerenciado externo (`:3306`) |
| `garage` + `garage-init` | Object Storage distribuído compatível com AWS S3 (assinatura SigV4) para armazenamento binário de fotos de veículos e avatares de perfil | `dxflrs/garage:v1.0.1` (Rust) + Alpine | Não (`expose :3900` — via proxy `/storage/`) |
| `Stripe` / `Mailtrap` | Serviços SaaS externos: Checkout recorrente, Customer Portal e Webhooks assinados (Stripe); disparo transacional de e-mails de reset (Mailtrap) | APIs HTTPS externas (`Stripe.net` / HTTP) | Webhook `POST` → Nginx → `backend` |

## 3.2 Onde roda

A aplicação roda na mesma máquina virtual Linux e na mesma stack gerenciada pelo **Portainer** (`portainer.lapps.studio`) utilizadas desde o primeiro bimestre, respondendo sob o domínio público `https://gabriel-assis-isw055.lapps.studio` (encaminhado pelo proxy reverso da infraestrutura para a porta host `8208` do container `frontend`). A stack puxa o código diretamente do repositório público do produto (github.com/Gabriel-Assis-22/Atividade_03_Claude, branch `main`, arquivo `docker-compose.yml`).

Todos os serviços de longa duração utilizam política `restart: unless-stopped`, enquanto o container auxiliar `garage-init` executa em modo **one-shot** (`restart: "no"`) apenas para inicializar o layout do nó e provisionar o bucket sobre os volumes persistentes `garage_meta` e `garage_data`. O `docker-compose.yml` não fixa travas rígidas de `mem_limit` por container, delegando o escalonamento de memória ao daemon Docker da VM; para preservar consumo enxuto na máquina compartilhada, optou-se deliberadamente por imagens base `alpine/aspnet:10.0` e pelo binário em Rust do `Garage S3` (~35 MB de RAM em operação, frente a ~250 MB do MinIO tradicional).

Nenhum segredo ou credencial é versionado no Git. As configurações são injetadas exclusivamente pelas variáveis de ambiente da stack no Portainer (documentadas em `.env.example` apenas pelos nomes):

| Serviço | Nomes das variáveis de ambiente declaradas (sem valores) |
|---|---|
| `backend` | `DB_HOST`, `DB_PORT`, `DB_USER`, `DB_PASSWORD`, `DB_NAME`, `JWT_SECRET`, `AUTH_SERVICE_URL`, `LOG_SERVICE_URL`, `FRONTEND_URL`, `ASPNETCORE_URLS`, |

| Serviço | Nomes das variáveis de ambiente declaradas (sem valores) |
|---|---|
| | `GARAGE_ENDPOINT`, `GARAGE_ACCESS_KEY`, `GARAGE_SECRET_KEY`, `GARAGE_BUCKET_NAME`, `GARAGE_PUBLIC_BASE_URL`, `STRIPE_SECRET_KEY`, `STRIPE_WEBHOOK_SECRET`, `STRIPE_PRICE_ID` |
| `auth-service` | `DB_HOST`, `DB_PORT`, `DB_USER`, `DB_PASSWORD`, `DB_NAME`, `JWT_SECRET`, `MAILTRAP_API_TOKEN`, `MAILTRAP_INBOX_ID`, `FRONTEND_URL`, `LOG_SERVICE_URL`, `ASPNETCORE_URLS` |
| `log-service` | `REDIS_CONNECTION`, `JWT_SECRET`, `ASPNETCORE_URLS` |
| `garage-init` | `GARAGE_ACCESS_KEY`, `GARAGE_SECRET_KEY`, `GARAGE_BUCKET_NAME` |

## 3.3 Fluxo de uma requisição

Para ilustrar o caminho ponta a ponta através dos componentes da stack, considera-se a operação central do negócio: **um passageiro autenticado solicita a reserva de um assento em uma carona disponível** (`POST /api/caronas/{id}/reservar`).

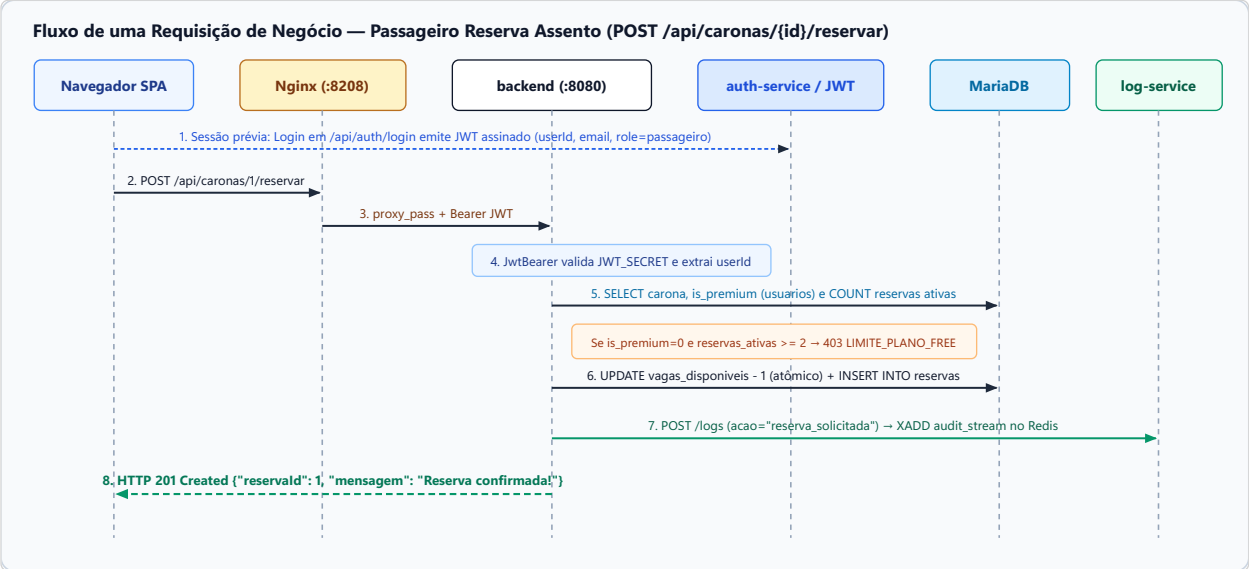



Figura 2: Fluxo completo de requisição: reserva de assento em carona atravessando Nginx, Backend, JWT, MariaDB e Redis Streams

1. **Origem no Navegador:** O estudante logado na SPA Angular clica em **"Reservar Vaga no UniRide"**. O interceptador HTTP (`jwt.interceptor.ts`) anexa automaticamente o cabeçalho `Authorization: Bearer <token>` emitido no login pelo `auth-service`.
2. **Entrada pelo Domínio e Proxy Reverso:** A requisição HTTPS chega ao subdomínio público, alcança a porta `:8208` do container `frontend` e é roteada pela regra `location /api/` do Nginx para `http://backend:8080/api/caronas/{id}/reservar`, preservando `X-Real-IP` e `X-Forwarded-For`.
3. **Autenticação e Papel (RBAC):** O middleware `JwtBearer` do `backend` valida localmente a assinatura criptográfica HMAC-SHA256 do token usando `JWT_SECRET` (Padrão B — Claims no Token) e extrai de forma confiável o identificador `CurrentUserId` e a claim `role`.
4. **Regras de Domínio e Transação no MariaDB:** O caso de uso `ReservaUseCases.ReservarAssentoAsync` consulta o `MariaDB` via `Dapper` para: (a) confirmar que a carona existe, está `disponivel` e possui vagas; (b) impedir que o motorista reserve a própria viagem; (c) barrar reserva duplicada (`uk_reserva_carona_passageiro`); e (d) verificar a coluna `is_premium` do passageiro em `usuarios` — se for plano gratuito (`is_premium = 0`) e já possuir 2 reservas confirmadas,

interrompe com `HTTP 403 Forbidden` (`LIMITE_PLANO_FREE`). Caso aprovado, executa o decremento condicional atômico `UPDATE caronas SET vagas_disponiveis = vagas_disponiveis - 1 WHERE id = @Id AND vagas_disponiveis > 0` e insere o registro em `reservas`.

5. **Auditoria Assíncrona e Resposta:** O `AuditClient` envia `POST http://log-service:8082/logs` com o evento `reserva_solicitada` (ou `bloqueio_limite_plano`), persistido no Redis via `XADD audit_stream *`, e o `CaronasController` devolve `HTTP 201 Created` à SPA, que atualiza o contador de assentos na tela.

## 3.4 Fluxo de assinatura

A monetização do **Plano UniRide Pro** (R$ 14,90/mês) opera em regime de assinatura recorrente (`Mode = "subscription"`) processada integralmente pelo Stripe em modo de teste:

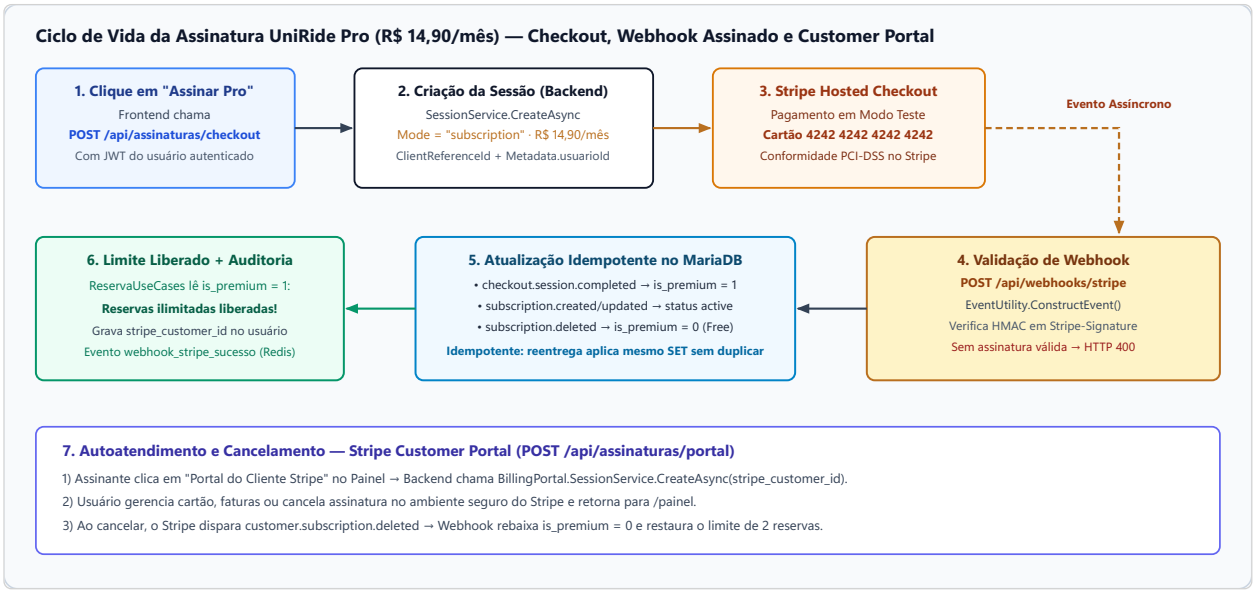



Figura 3: Ciclo de vida da assinatura recorrente: Checkout Session, validação criptográfica de Webhook, MariaDB e Customer Portal

- **Início do Checkout (`POST /api/assinaturas/checkout`):** A partir do botão **"Assinar UniRide Pro"** (ou do botão **"Fazer Upgrade"** exibido quando o limite gratuito bloqueia a 3ª reserva), o `StripePaymentService` cria uma `Checkout.Session` com `Mode = "subscription"`, valor mensal de R$ 14,90 (`STRIPE_PRICE_ID` ou `PriceData` recorrente em BRL), injetando o `usuarioId` tanto em `ClientReferenceId` quanto nos dicionários `Metadata` da sessão e da `SubscriptionData`. O navegador é redirecionado para a página hospedada pelo Stripe (onde se utiliza o cartão de teste `4242 4242 4242 4242`).
- **Recepção e Validação Criptográfica do Webhook (`POST /api/webhooks/stripe` e `/api/payments/webhook`):** Após a cobrança, o Stripe dispara uma requisição assíncrona ao backend contendo o cabeçalho `Stripe-Signature`. O servidor exige a presença desse cabeçalho (recusando chamadas sem assinatura com `HTTP 400 Bad Request`) e executa `EventUtility.ConstructEvent(jsonPayload, stripeSignature, _webhookSecret)`, que recalcula o HMAC-SHA256 do payload bruto contra `STRIPE_WEBHOOK_SECRET` e valida a janela temporal contra ataques de **replay**.
- **Transição de Estado no Banco e Idempotência:** O `HandleStripeWebhookUseCase` processa os eventos `checkout.session.completed`, `customer.subscription.created` e `customer.subscription.updated` (marcando `is_premium = 1` quando `status` é `active` ou `trialing` e salvando `stripe_customer_id` na tabela `usuarios`) e `customer.subscription.deleted` (rebaixando `is_premium = 0`). Caso o Stripe entregue **o mesmo evento duas vezes** (comportamento padrão

**at-least-once** de webhooks), a operação `UPDATE usuarios SET is_premium = @IsPremium, stripe_customer_id = @StripeCustomerId WHERE id = @Id` é naturalmente **idempotente**: o estado resultante permanece idêntico, sem duplicação de registros nem inconsistência de saldo, retornando `200 OK` (`{"received": true}`).

- **Cancelamento pelo Portal do Cliente (`POST /api/assinaturas/portal`):** No painel do usuário, o assinante aciona `BillingPortal.SessionService.CreateAsync` passando seu `stripe_customer_id`. No portal oficial do Stripe, ele pode cancelar a renovação ou trocar o meio de pagamento; o cancelamento emite `customer.subscription.deleted`, revertendo o usuário automaticamente às cotas do plano gratuito.

## 3.5 Modelo de dados do domínio

O esquema relacional no `MariaDB` é gerenciado de forma automatizada e idempotente na subida dos containers pelo `FluentMigrator` (migrações `_001` a `_009` no `backend` e `_001` a `_004` no `auth-service`, com tabelas de controle de versão segregadas). A ligação entre identidade, plano comercial e assinatura no Stripe reside diretamente na tabela `usuarios` por meio das colunas `role`, `is_premium` (`0` = Passageiro Free; `1` = UniRide Pro) e `stripe_customer_id` (`cus_...`).

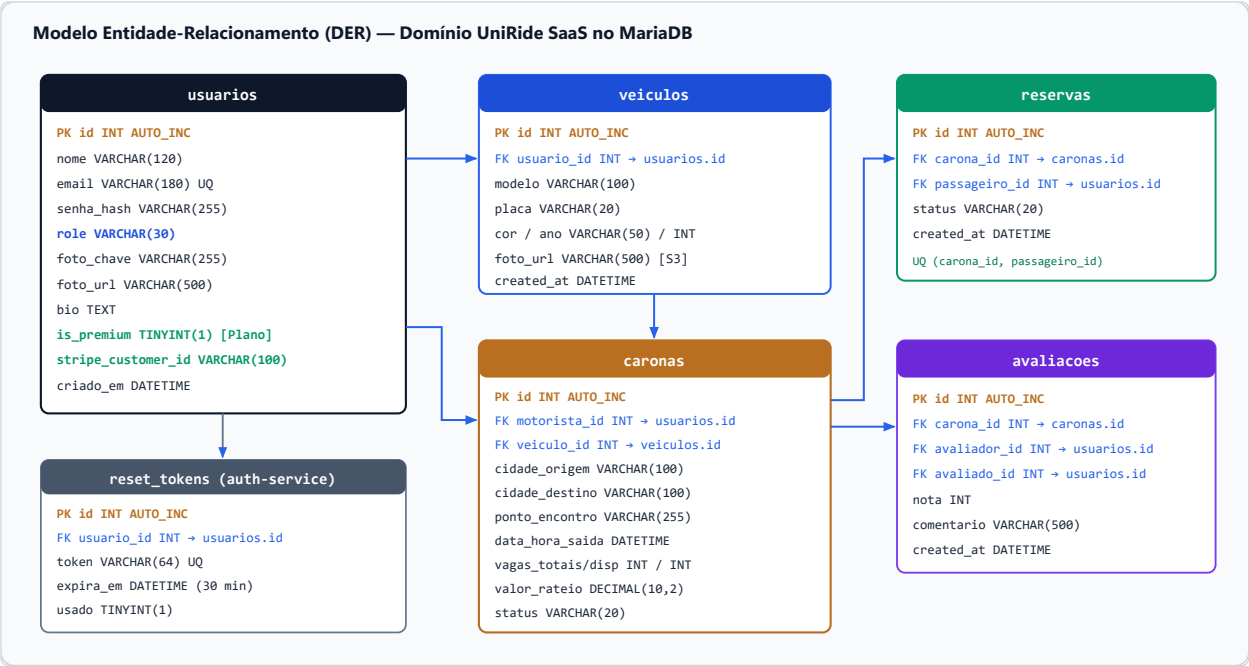



Figura 4: Diagrama Entidade-Relacionamento (DER) das tabelas do domínio UniRide no MariaDB

| Tabela | PK / Chaves Estrangeiras | Colunas principais e papel no domínio |
|---|---|---|
| `usuarios` | `PK id` | `nome`, `email` (UNIQUE), `senha_hash`, `role` (`passageiro`, `motorista`, `admin`), `is_premium` (plano ativo), `stripe_customer_id` (vínculo Stripe), `foto_chave`, `foto_url`, `bio`, `criado_em` |
| `veiculos` | `PK id`<br>`FK usuario_id → usuarios.id` | `modelo`, `placa`, `cor`, `ano`, `foto_url` (chave pública no Garage S3), `created_at`. Exclusivo de condutores |

| Tabela | PK / Chaves Estrangeiras | Colunas principais e papel no domínio |
|---|---|---|
| `caronas` | `PK id`<br>`FK motorista_id → usuarios.id`<br>`FK veiculo_id → veiculos.id` | `cidade_origem`, `cidade_destino`, `ponto_encontro`, `data_hora_saida`, `vagas_totais`, `vagas_disponiveis`, `valor_rateio`, `status` (`disponivel`, `cancelada`). Índice composto `idx_caronas_busca` |
| `reservas` | `PK id`<br>`FK carona_id → caronas.id`<br>`FK passageiro_id → usuarios.id` | `status` (`confirmada`, `cancelada`), `created_at`. Constraint única `uk_reserva_carona_passageiro` evitando dupla reserva na mesma viagem |
| `avaliacoes` | `PK id`<br>`FK carona_id → caronas.id`<br>`FK avaliador_id, avaliado_id` | `nota`, `comentario`, `created_at`. Reputação mútua entre condutores e passageiros após viagens |
| `reset_tokens` | `PK id`<br>`FK usuario_id → usuarios.id` | `token` (32 bytes hex único), `criado_em`, `expira_em` (+30 min), `usado`. Mantida pelo `auth-service` |

### 3.6 Segurança

O modelo de segurança combina controle de acesso baseado em papéis (**RBAC**) avaliado no servidor via atributos `[Authorize(Roles = "...")]` com verificações explícitas de posse de recurso (**Anti-IDOR**) na camada de casos de uso:

| Ação no sistema | `admin` | `motorista` | `passageiro` |
|---|---|---|---|
| Buscar e visualizar feed de caronas disponíveis (`GET /api/caronas`) | sim | sim | sim |
| Reservar assento em carona de terceiros (`POST /api/caronas/{id}/reservar`) | sim | sim | sim (cota plano) |
| Cadastrar veículo e enviar foto ao S3 (`POST /api/veiculos`) | sim | sim | não (`403`) |
| Publicar nova rota de carona (`POST /api/caronas`) | sim | sim | não (`403`) |
| Cancelar carona ou usar veículo pertencente a outro usuário | moderação | não (`403`) | não (`403`) |
| Alterar perfil/foto de outro usuário ou cancelar reserva alheia | não (`403`) | não (`403`) | não (`403`) |
| Consultar métricas financeiras de MRR (`GET /api/assinaturas/metricas`) | sim | não (`403`) | não (`403`) |
| Auditar trilha de eventos do Redis Streams (`GET /api/logs`) | sim | não (`403`) | não (`403`) |

**Isolamento por usuário e proteção Anti-IDOR:** Em todos os endpoints autenticados (`VeiculosController`, `CaronasController`, `ReservasController`, `ProfileController`, `PaymentsController`), a identidade operante jamais é lida do corpo JSON ou de parâmetros manipuláveis pelo cliente, mas sim extraída criptograficamente das claims do JWT (`CurrentUserId`). Consultas de listagem pessoal (`GET /api/veiculos/meus`, `GET /api/caronas/minhas`, `GET /api/reservas/minhas`) filtram obrigatoriamente por `WHERE usuario_id = @CurrentUserId`. Nas mutações (`CriarCaronaAsync`, `CancelarCaronaAsync`, `CancelarReservaAsync`, `UpdateProfileUseCase`), o caso de uso confere no banco se o recurso pertence ao `solicitanteId`; qualquer divergência lança `UnauthorizedAccessException`, respondida com `HTTP 403 Forbidden` e registrada imediatamente no `log-service` (`bloqueio_seguranca` / `tentativa_idor_bloqueada`).

**Gestão de segredos:** Chaves privadas (`JWT_SECRET`, `DB_PASSWORD`, `STRIPE_SECRET_KEY`, `STRIPE_WEBHOOK_SECRET`, `GARAGE_SECRET_KEY`, `MAILTRAP_API_TOKEN`) residem exclusivamente nas variáveis de ambiente da stack no Portainer e no arquivo `.env` local ignorado pelo `.gitignore`. No repositório público consta apenas `.env.example` com chaves ilustrativas sem valor real.

## 3.7 Observabilidade, CI/CD e documentação da API

Entre as entregas extras opcionais da disciplina, foi implementada integralmente a **Documentação Interativa de API com Swagger / OpenAPI 3.0** (`Swashbuckle.AspNetCore`), acessível na rota pública `/swagger/` (via proxy reverso Nginx) e versionada estaticamente em `docs/openapi.json`, incluindo suporte nativo ao esquema de segurança `Bearer JWT` para teste direto de todos os contratos de Caronas, Veículos, Reservas, Assinaturas Stripe, Perfil e Logs. Na vertente de observabilidade de negócio e segurança, o sistema conta com o painel administrativo de indicadores SaaS (`GET /api/assinaturas/metricas` calculando usuários totais, Free, Pro e MRR) e com a trilha centralizada de eventos no `log-service` / Redis Streams. As atividades extras opcionais de pipeline CI/CD no GitHub Actions e endpoints dedicados de métricas Prometheus não foram implementadas neste ciclo.

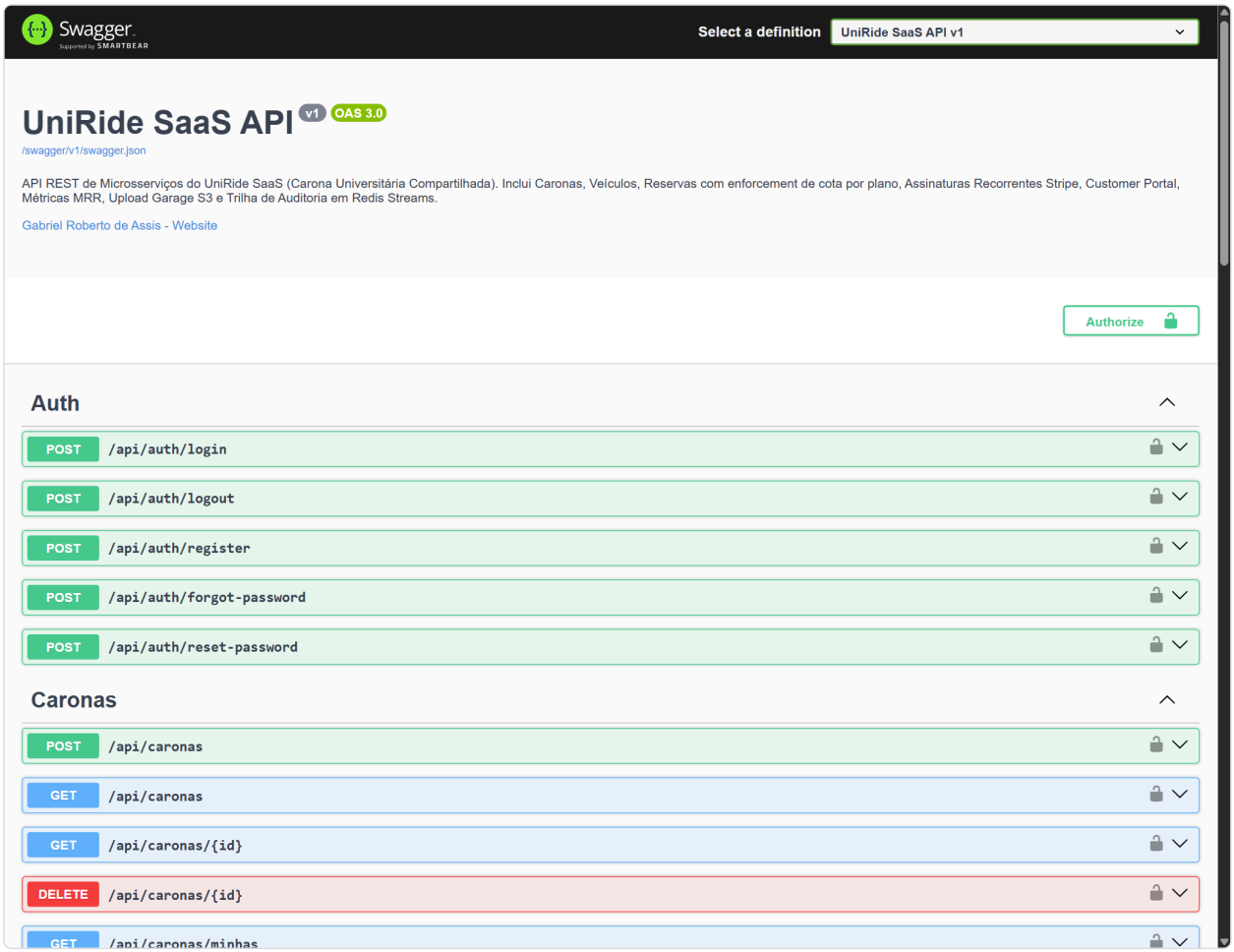



Figura 5: Especificação OpenAPI 3.0 / Swagger UI do UniRide SaaS em execução na rota /swagger

## 3.8 Camadas IaaS, PaaS e SaaS do projeto

Aplicando ao produto final UniRide a classificação arquitetural de computação em nuvem estudada na Atividade 8, as responsabilidades operacionais distribuem-se nas três camadas abaixo:

| Camada | O que é no projeto UniRide | Quem gerencia |
|---|---|---|
| **IaaS** | Máquina virtual Linux provisionada no cluster da disciplina (vCPU, memória RAM, armazenamento em disco virtual, rede e roteamento de borda para `lapps.studio`) | Infraestrutura da Fatec / Prof. Allan Siriani (SO e daemon sob uso da stack) |
| **PaaS** | Ambiente de orquestração **Docker Engine + Portainer**, banco relacional compartilhado **MariaDB** (`:3306`) e containers de plataforma (`redis:7-alpine`, `garage:v1.0.1`, runtime `aspnet:10.0` e `nginx:alpine`) sobre a rede `app-network` | Plataforma base pela Fatec/Portainer; definição de `Dockerfiles`, `docker-compose.yml` e migrações pelo aluno |
| **SaaS** | 1) **O próprio UniRide**, entregue pronto via navegador ao estudante universitário final sem instalação local;<br>2) **Serviços SaaS de terceiros consumidos via API: Stripe** | UniRide operado pelo aluno desenvolvedor; |

| Camada | O que é no projeto UniRide | Quem gerencia |
|---|---|---|
| | (pagamentos, portal e webhooks), **Mailtrap** (e-mails transacionais) e **GitHub** (hospedagem Git e issues) | Stripe, Mailtrap e GitHub gerenciados integralmente pelos respectivos provedores |

# 4 Quadro de entregas

A tabela a seguir consolida automaticamente os prazos planejados e os registros temporais reais de entrega (data e hora no fuso `America/Sao_Paulo`, UTC-3) apurados nas fichas individuais da Seção 5:

| Nº | Atividade | Data planejada | Data realizada | Situação |
|---|---|---|---|---|
| 7 | Plano Premium — cobrança de verdade com Stripe | 09/10/2026 | 19/10/2026 20:37 | entregue com atraso |
| 8 | SaaS, PaaS, IaaS — e a documentação que o projeto nunca teve | 16/10/2026 | 19/10/2026 20:59 | entregue com atraso |
| 9 | Fork e plano de negócio | 23/10/2026 | 22/10/2026 20:32 | entregue |
| 10 | Seu negócio no ar — o domínio no lugar dos filmes | 06/11/2026 | 22/10/2026 13:38 | entregue |
| 11 | SaaS completo — planos, limites e assinatura com Stripe | 13/11/2026 | 22/10/2026 13:51 | entregue |

# 5 Atividades realizadas

Nesta seção são apresentadas as fichas detalhadas das cinco atividades obrigatórias do 2º bimestre (Atividades 7 a 11), contendo os hashes e links diretos verificáveis no GitHub, os horários exatos extraídos do controle de versão (fuso `America/Sao_Paulo`, UTC-3), as capturas comprobatórias de entrega (página web do GitHub acompanhada da verificação `git log` com data e hora explícitas) e o relato técnico das soluções adotadas.

## 5.1 Atividade 7 — Plano Premium — cobrança de verdade com Stripe

| | |
|---|---|
| **Descrição** | Plano pago em modo de teste: Checkout Session do Stripe e webhook marcando o usuário como premium. |
| **Data planejada** | 09/10/2026 |
| **Data realizada** | 19/10/2026 20:37 |
| **Situação** | entregue com atraso |
| **Evidência** | GitHub — commit 59f4155 (19/10/2026 20:37) + fluxo de upgrade e bloqueio de cota do plano gratuito no UniRide |
| **Link** | https://github.com/Gabriel-Assis-22/Atividade_02_Claude/commit/59f4155b39f53bc7b8ade40b791db3016f4fa811 |

**O que foi feito.** Implementou-se a integração ponta a ponta com o provedor de pagamentos **Stripe** (em modo de teste) utilizando o SDK `Stripe.net`, consolidada posteriormente no domínio do **UniRide SaaS**. No banco `MariaDB`, a `Migration_004_AddPremiumColumnsToUsuarios` adicionou as colunas `is_premium` e `stripe_customer_id` à tabela `usuarios`. No `backend` (.NET 10), foram construídos o `PaymentsController`, o serviço `StripePaymentService` para geração de `Checkout.Session` hospedada e o caso de uso `HandleStripeWebhookUseCase` para recepção assíncrona do evento `checkout.session.completed` com validação criptográfica HMAC-SHA256 via cabeçalho `Stripe-Signature`, promovendo o usuário ao plano pago sem transitar dados sensíveis de cartão pelo servidor (conformidade PCI-DSS) e desbloqueando o limite do plano gratuito (`HTTP 403 Forbidden` na tentativa de exceder a cota gratuita). A entrega inicial foi concluída em `19/10/2026 20:37` (10 dias após a data planejada de `09/10/2026`) em razão do acúmulo da consolidação do relatório da P1 na semana imediatamente anterior.

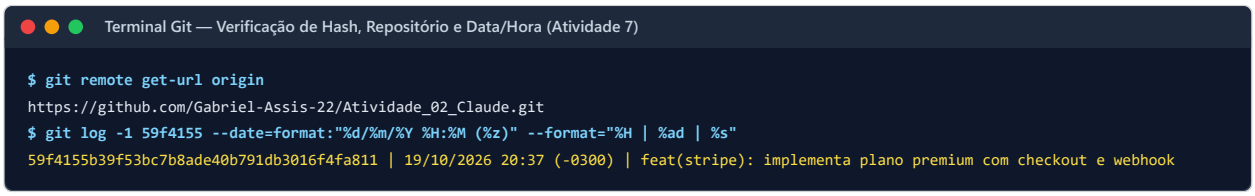



Figura 6: Atividade 7 — verificação no terminal Git com repositório, hash completo (59f4155) e data/hora exatas (19/10/2026 20:37 BRT)

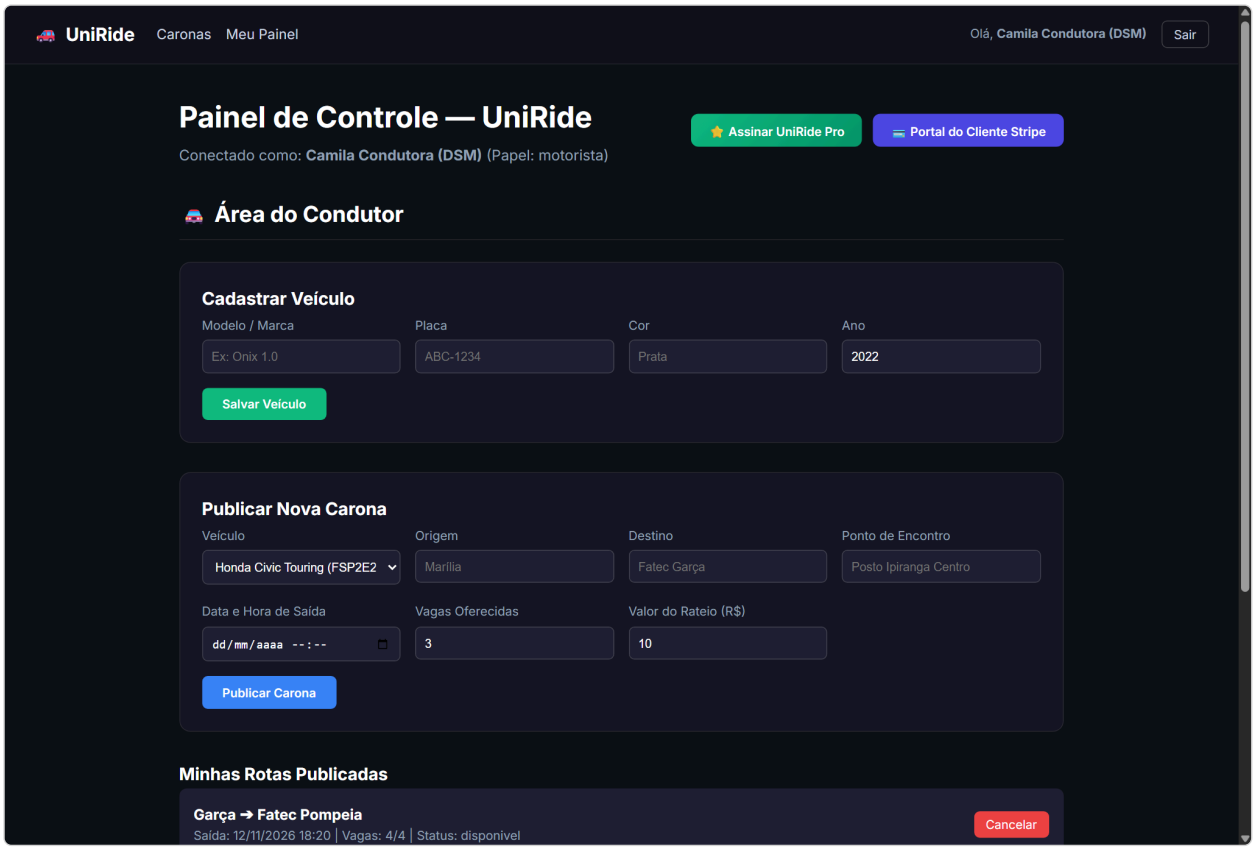



Figura 7: Atividade 7 — painel autenticado do UniRide exibindo os controles de upgrade (“Assinar UniRide Pro”) e autoatendimento Stripe

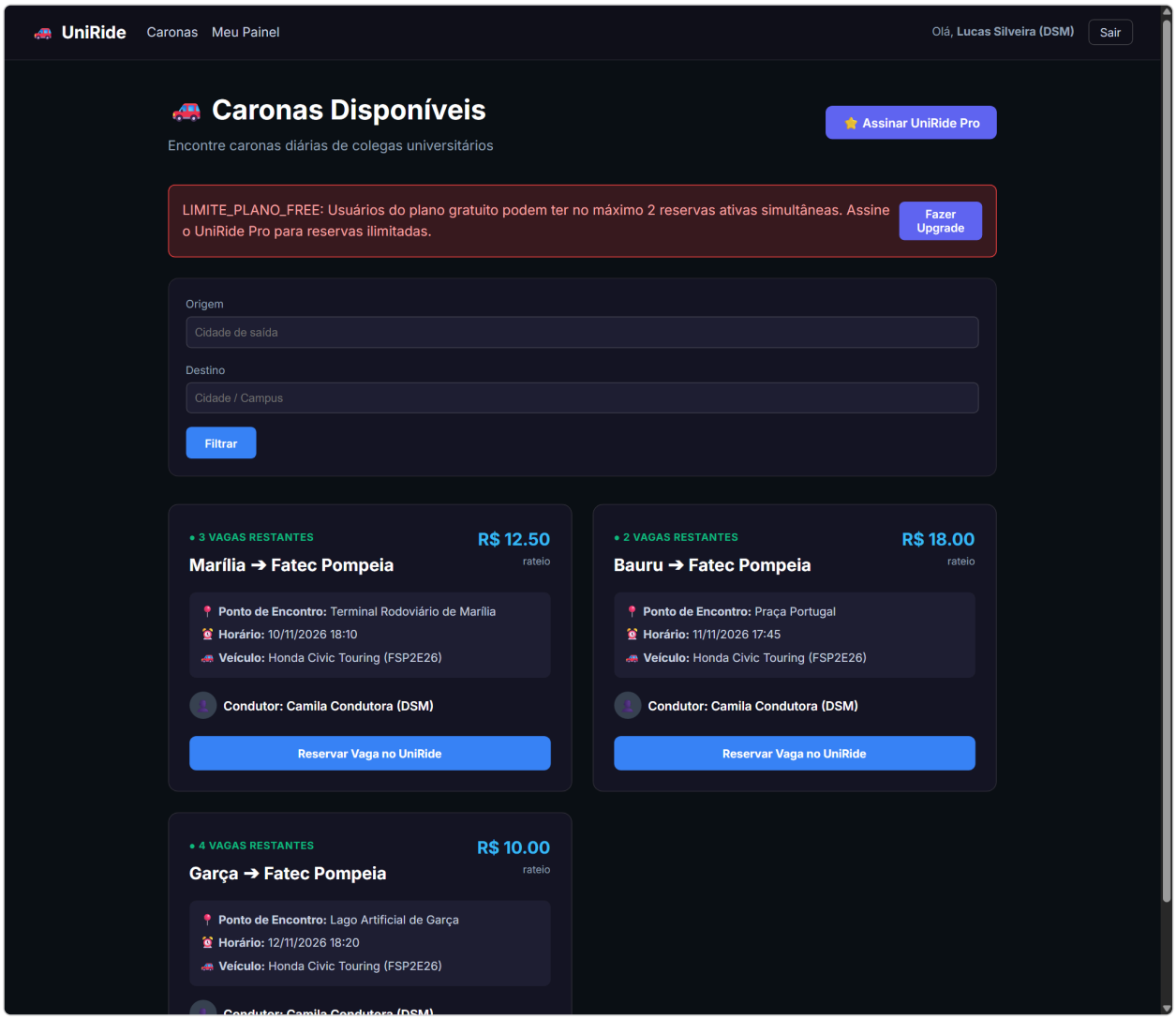



Figura 8: Atividade 7 — resultado da diferenciação entre plano gratuito e pago: bloqueio HTTP 403 na cota gratuita e CTA de upgrade

**Dificuldades e como foram resolvidas.** A principal dificuldade técnica ocorreu na validação criptográfica do webhook: qualquer leitura prévia ou **model binding** automático do ASP.NET Core alterava o stream de bytes e invalidava o hash calculado por `EventUtility.ConstructEvent`. O problema foi resolvido lendo o corpo bruto diretamente via `StreamReader(HttpContext.Request.Body)` em endpoint `[AllowAnonymous]` dedicado e validando obrigatoriamente o segredo `STRIPE_WEBHOOK_SECRET` antes de acionar o caso de uso.

## 5.2 Atividade 8 — SaaS, PaaS, IaaS — e a documentação que o projeto nunca teve

| | |
|---|---|
| **Descrição** | Camadas de nuvem do próprio projeto e documentação reversa: especificação, arquitetura, regras de negócio e modelo de dados. |
| **Data planejada** | 16/10/2026 |
| **Data realizada** | 19/10/2026 20:59 |
| **Situação** | entregue com atraso |
| **Evidência** | GitHub — commits 22af01e (20:54) e 89f32ef (19/10/2026 20:59) + documentação reversa, DER e Swagger UI do UniRide |
| **Link** | https://github.com/Gabriel-Assis-22/Atividade_02_Claude/commit/89f32ef54b2749581776d9d0d7df27cebb754985 |

**O que foi feito.** Realizou-se o levantamento completo por engenharia reversa de todos os microsserviços, fluxos de dados, regras de negócio implícitas e tabelas relacionais do projeto, consolidando o manual arquitetural em `DOCUMENTACAO.md` (commit `22af01e` às `20:54`) e adicionando ao `README.md` principal o quadro classificatório das camadas de computação em nuvem (**IaaS**, **PaaS** e **SaaS**) aplicadas à própria stack do projeto, com referência cruzada direta para a documentação reversa (commit `89f32ef` às `20:59`). Na mesma noite (`19/10/2026 22:43`, commit `8cf880e`), implementou-se também a entrega extra opcional de documentação interativa **OpenAPI 3.0 / Swagger UI** (`Swashbuckle.AspNetCore`) nos serviços `.NET`, atualizada integralmente para os endpoints do domínio UniRide. A atividade registrou atraso de três dias em relação ao prazo de `16/10/2026` por depender da finalização prévia das tabelas e rotas financeiras da Atividade 7 para retratar o sistema completo sem lacunas.

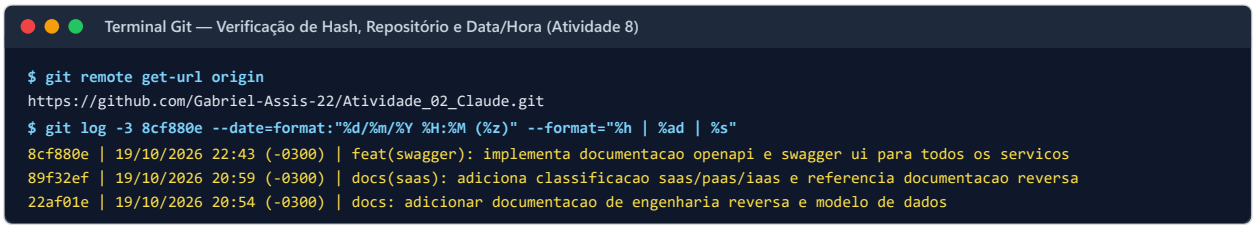



Figura 9: Atividade 8 — verificação no terminal Git com repositório, hashes (22af01e, 89f32ef, 8cf880e) e datas/horas exatas (19/10/2026 BRT)

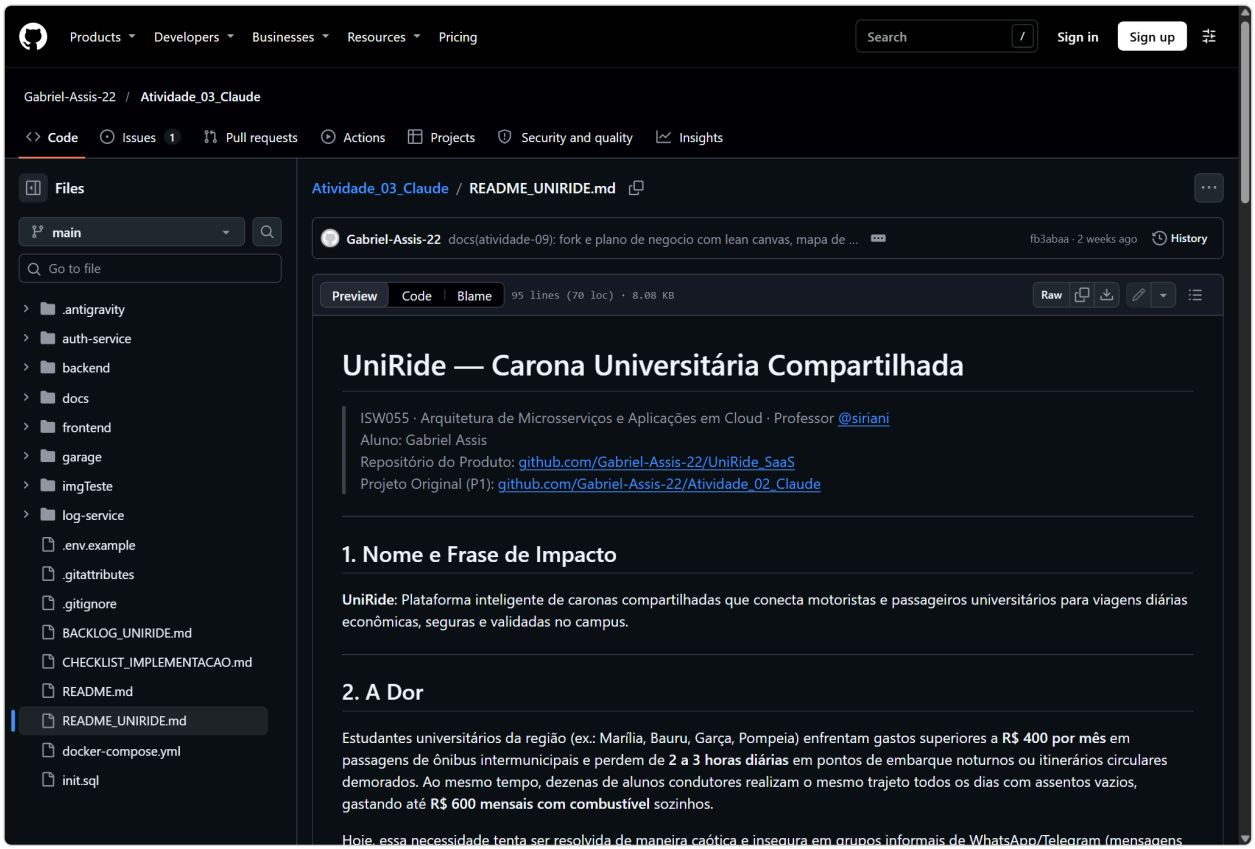



Figura 10: Atividade 8 — documentação estruturada e mapeamento de reaproveitamento arquitetural no repositório público do UniRide

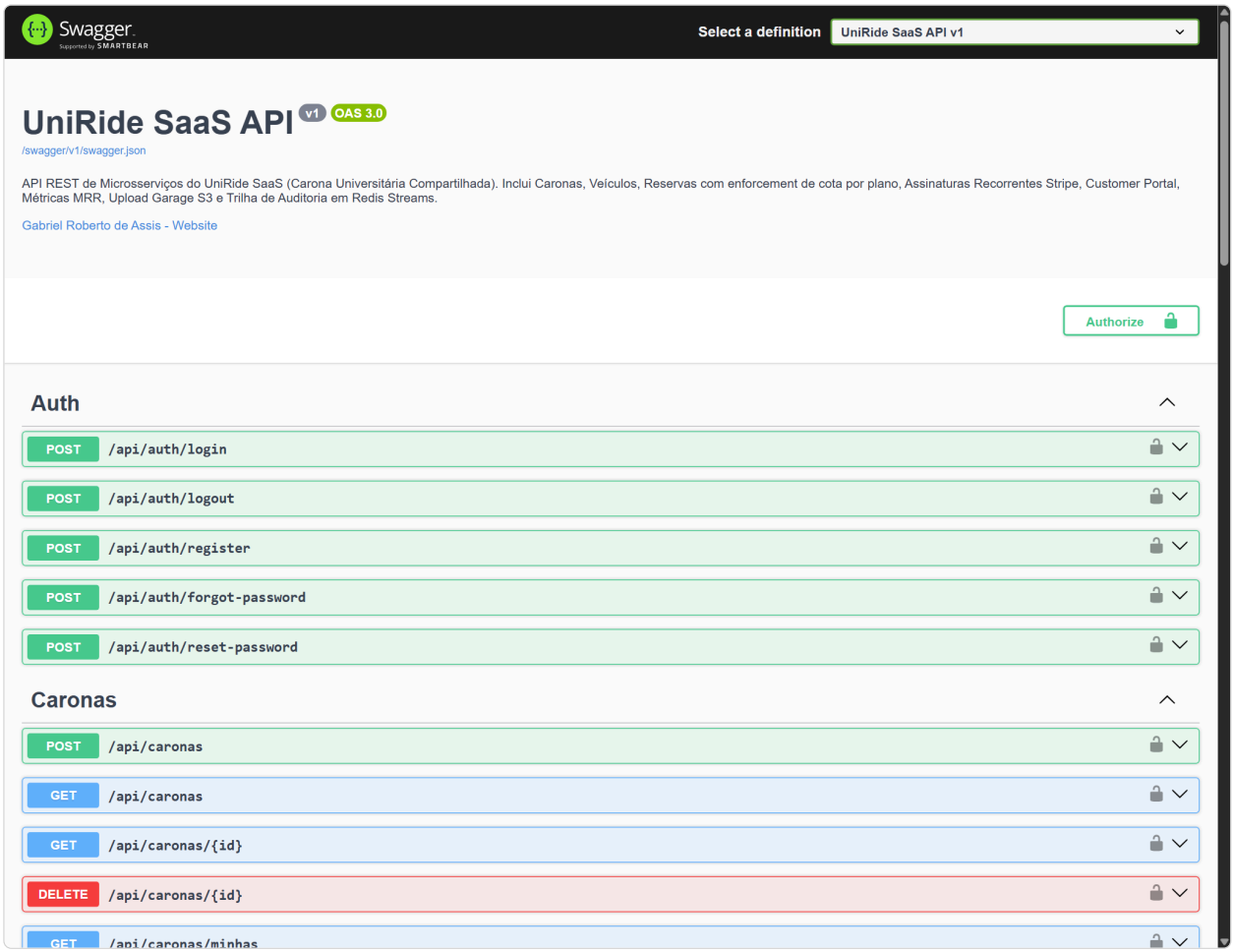



Figura 11: Atividade 8 — resultado da engenharia reversa: contratos da API documentados e testáveis no Swagger UI (/swagger)

**Dificuldades e como foram resolvidas.** Mapear fielmente fronteiras que haviam crescido organicamente em seis containers exigiu auditar cada `Dockerfile`, `nginx.conf` e migração FluentMigrator para distinguir com precisão o que pertencia à camada PaaS gerenciada pela Fatec/Portainer e o que constituía SaaS próprio ou SaaS de terceiros (Stripe, Mailtrap e GitHub). Essa auditoria serviu de alicerce direto para executar a transição limpa para o domínio de caronas universitárias.

## 5.3 Atividade 9 — Fork e plano de negócio

| | |
|---|---|
| **Descrição** | Repositório novo com o histórico do anterior, README do produto (dor, persona, Lean Canvas, modelo de receita, mapa de reaproveitamento) e backlog em issues. |
| **Data planejada** | 23/10/2026 |
| **Data realizada** | 22/10/2026 20:32 |
| **Situação** | entregue |
| **Evidência** | GitHub — Issue #1 mencionando @siriani (22/10/2026 20:32) + commit fb3abaa + README do produto e backlog |
| **Link** | https://github.com/Gabriel-Assis-22/Atividade_03_Claude/issues/1 |

**O que foi feito.** Criou-se o novo repositório público dedicado ao produto SaaS (github.com/Gabriel-Assis-22/Atividade_03_Claude) a partir da base consolidada do projeto anterior, estruturando a concepção de negócio do **UniRide — Carona Universitária Compartilhada** no commit `fb3abaa` (`22/10/2026 13:37`) e formalizando a entrega dentro do prazo por meio da abertura da **Issue #1** (`22/10/2026 20:32`) com menção direta a `@siriani`. Foram elaborados o documento executivo de produto (`README_UNIRIDE.md` / `README.md`) — contemplando problema real de deslocamento intermunicipal estudantil, personas primárias, **Lean Canvas** completo em 9 blocos, tabela de precificação recorrente (**Passageiro Free** vs **UniRide Pro** R$ 14,90/mês) e matriz de reaproveitamento técnico dos microsserviços — além do mapeamento das entregas de engenharia (`BACKLOG_UNIRIDE.md`).




Figura 12: Atividade 9 — Issue #1 aberta no repositório do produto mencionando `@siriani`

GitHub API & Git — Verificação de Repositório, Issue #1 (@siriani) e Data/Hora (Atividade 9)

```
$ git remote get-url origin -> https://github.com/Gabriel-Assis-22/Atividade_03_Claude.git
$ git log -1 fb3abaa --date=format:"%d/%m/%Y %H:%M (%z)" --format="%h | %ad | %s"
fb3abaa | 22/10/2026 13:37 (-0300) | docs(atividade-09): fork e plano de negocio com lean canvas, mapa de reapro
$ curl -s https://api.github.com/repos/Gabriel-Assis-22/Atividade_03_Claude/issues/1
Issue #1: "Novo Repo. Atividade 09 da aula de Claude." | Body: "@siriani"
created_at (UTC): 2026-10-22T23:32:29Z => Data/Hora local: 22/10/2026 20:32 BRT (-0300)
```

Figura 13: Atividade 9 — verificação via GitHub API e Git do timestamp exato da Issue #1 (22/10/2026 20:32 BRT) e do commit fb3abaa

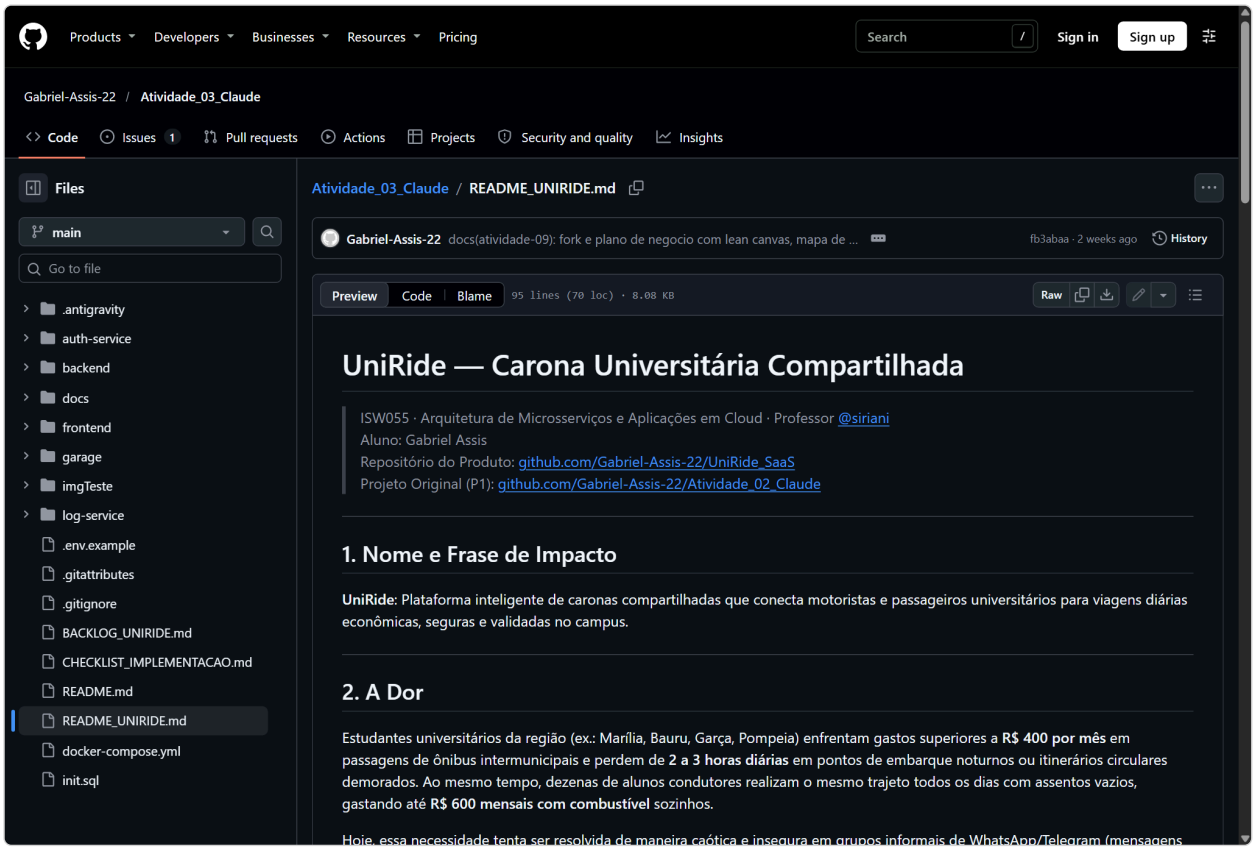



Figura 14: Atividade 9 — resultado: especificação de negócio do UniRide com Lean Canvas, modelo de receita e reaproveitamento

**Dificuldades e como foram resolvidas.** Na migração para o repositório `Atividade_03_Claude`, a cópia inicial dos arquivos consolidou a base do 1º bimestre em um commit raiz (`7dcbc4f`) em vez de `git push --mirror`; para preservar total transparência histórica, o repositório anterior (`Atividade_02_Claude`) foi mantido público e referenciado explicitamente na documentação, permitindo auditar todos os hashes originais de agosto a outubro sem perda de rastreabilidade.

## 5.4 Atividade 10 — Seu negócio no ar — o domínio no lugar dos filmes

| | |
|---|---|
| **Descrição** | Stack apontando para o repositório novo, tabelas e API do domínio, papéis do negócio, isolamento por usuário, eventos no log, upload e landing page. |
| **Data planejada** | 06/11/2026 |
| **Data realizada** | 22/10/2026 13:38 |
| **Situação** | entregue |
| **Evidência** | GitHub — commit a7b62c4 (22/10/2026 13:38) + prints da Landing Page, do Painel do Condutor e dos logs Anti-IDOR no Redis |
| **Link** | https://github.com/Gabriel-Assis-22/Atividade_03_Claude/commit/a7b62c482df12b305309814f70f51dae64354875 |

**O que foi feito.** Substituiu-se integralmente o domínio anterior pelo núcleo operacional do **UniRide** (commit `a7b62c4` em `22/10/2026 13:38`). No banco `MariaDB`, as migrações `_006` a `_009` criaram as tabelas `veiculos`, `caronas`, `reservas` e `avaliacoes`. No `auth-service` e no `backend`, foram configurados os 3 papéis do negócio (`passageiro`, `motorista` e `admin`), CRUD de veículos com upload autenticado de fotos no `Garage S3`, oferta e busca filtrada de rotas, reserva atômica de assentos (`vagas_disponiveis - 1` condicional), proteção estrita **Anti-IDOR** (`HTTP 403 Forbidden` ao tentar manipular carona, veículo ou reserva de terceiro) e publicação de eventos de domínio no `log-service` / Redis Streams (`carona_criada`, `reserva_solicitada`, `bloqueio_seguranca`). No `frontend` Angular 19, desenvolveu-se a Landing Page pública com proposta de valor, busca rápida e comparativo de planos, além do Feed de Caronas e Painel do Condutor/Passageiro.

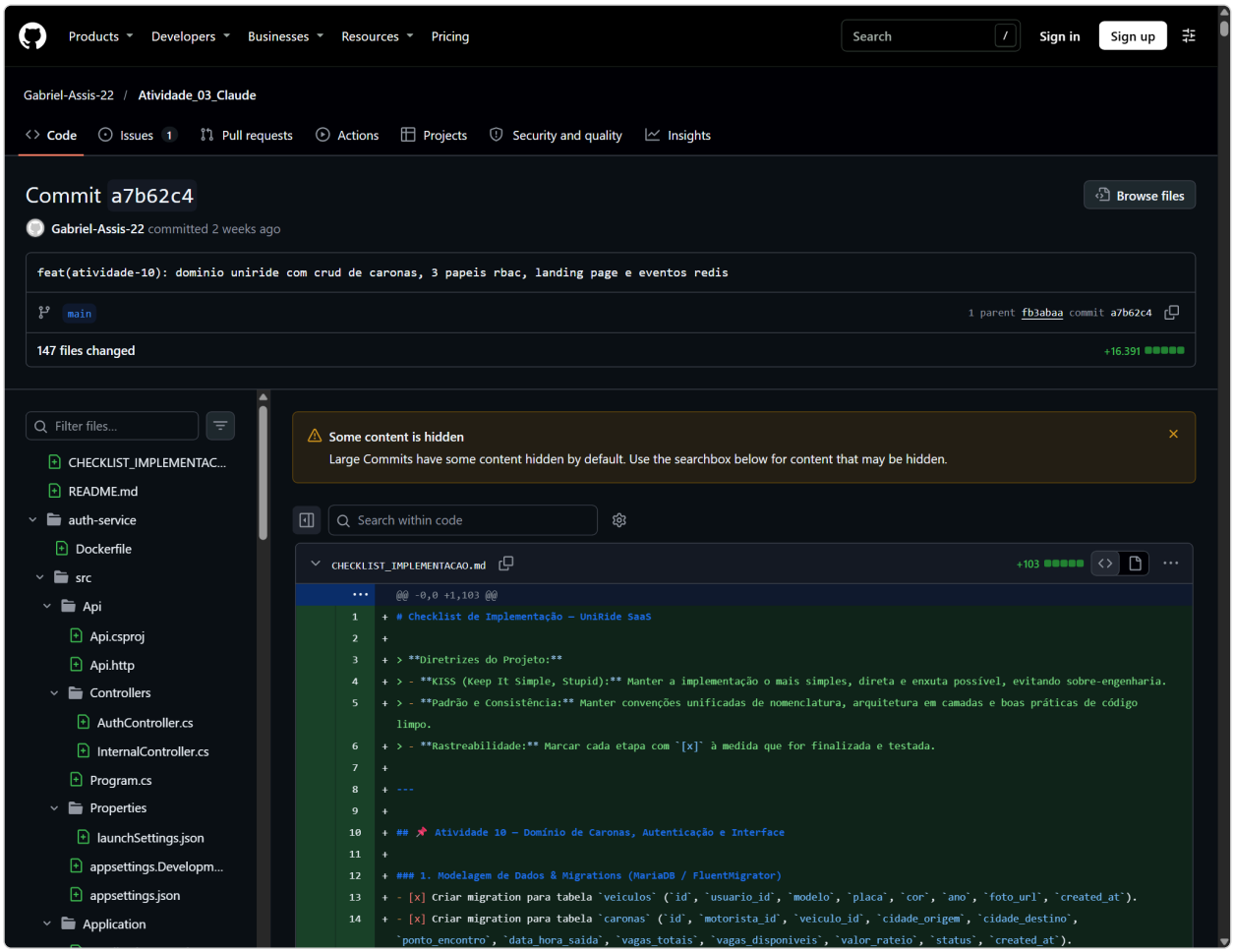



Figura 15: Atividade 10 — página do commit a7b62c4 no repositório do produto (Gabriel-Assis-22/Atividade_03_Claude)

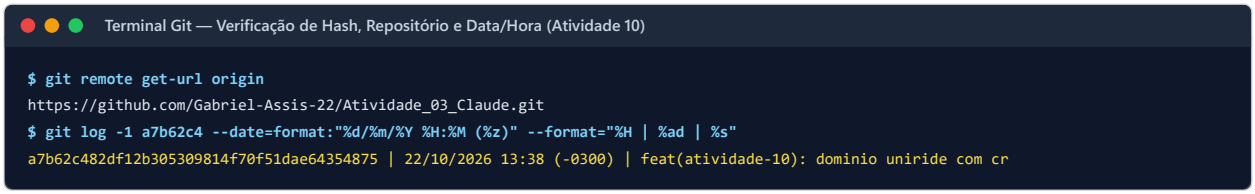



Figura 16: Atividade 10 — verificação no terminal Git com repositório, hash completo e data/hora exatas (22/10/2026 13:38 BRT)

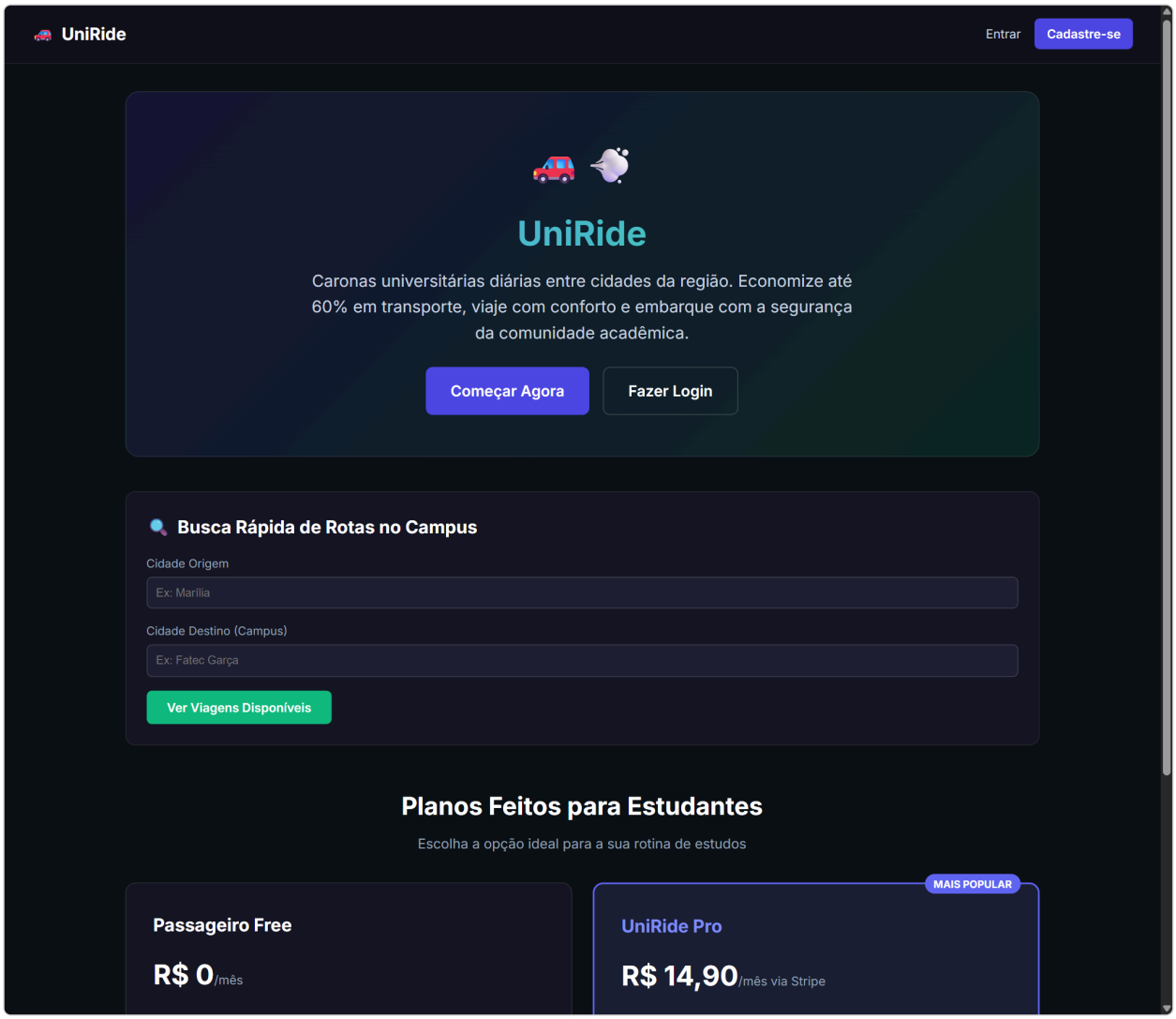



Figura 17: Atividade 10 — resultado (1/3): Landing Page pública do UniRide com busca de rotas e comparativo de planos Free vs Pro

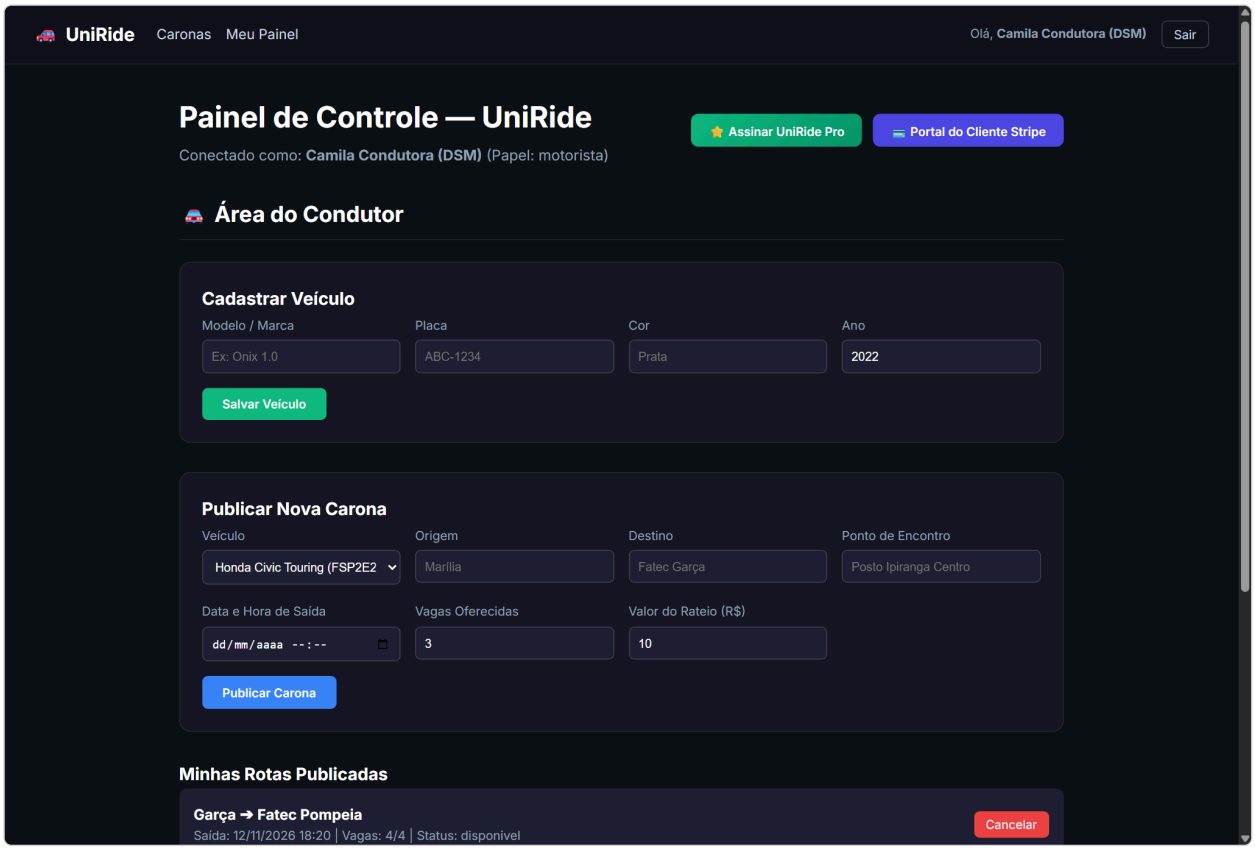



Figura 18: Atividade 10 — resultado (2/3): Painel do Condutor autenticado (papel motorista) com cadastro de veículo, publicação de carona e isolamento das próprias rotas

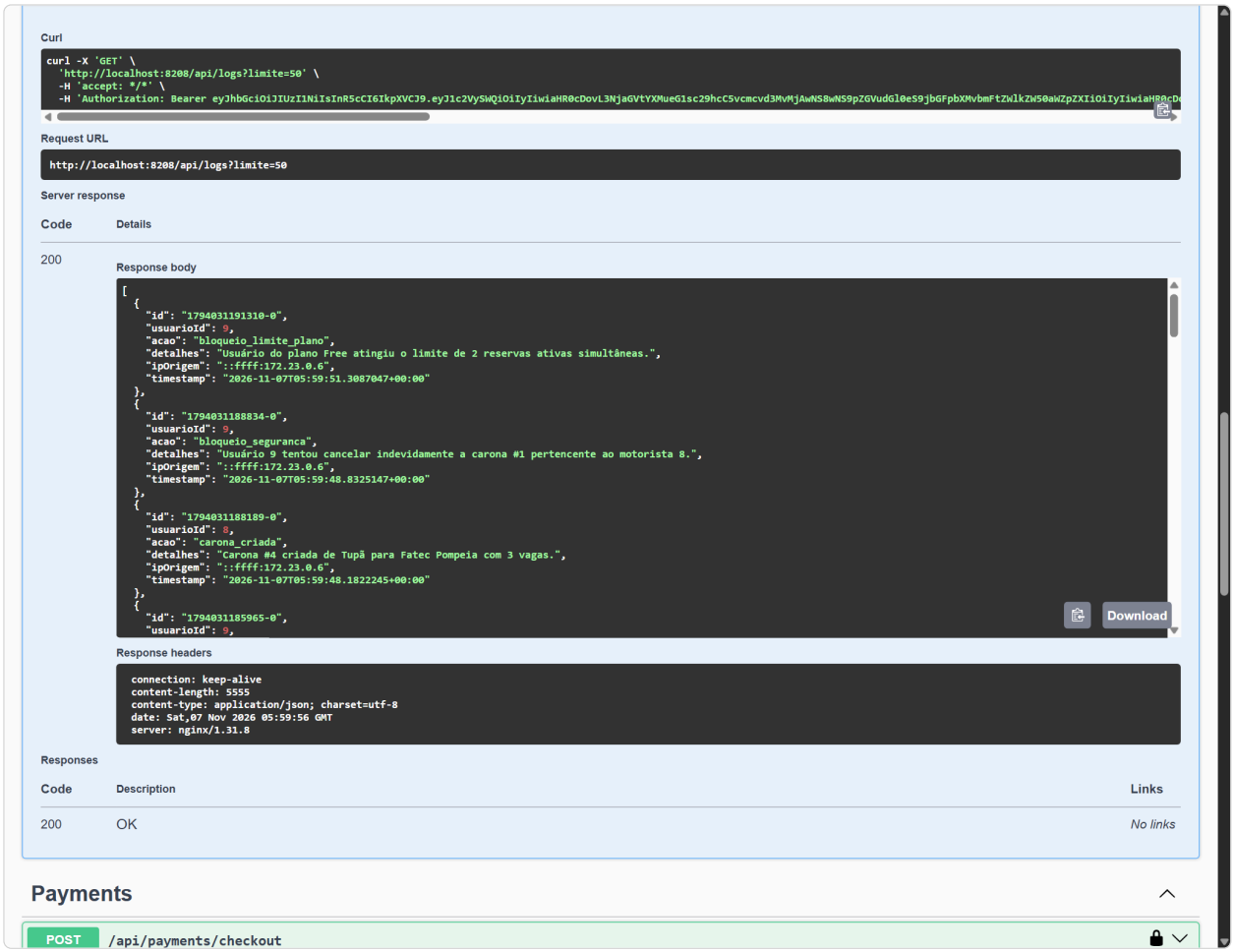



Figura 19: Atividade 10 — resultado (3/3): consulta administrativa GET /api/logs exibindo eventos de negócio (carona_criada) e bloqueio Anti-IDOR (bloqueio_seguranca) no Redis Streams

**Dificuldades e como foram resolvidas.** Ao injetar auditoria transversal nos casos de uso de caronas e reservas, surgiu uma referência circular/indevida entre `Application` e `Infrastructure` durante o `dotnet publish` em container limpo, além da interceptação de assets `.js` do Swagger pela regra estática do Nginx. Ambos foram solucionados movendo o contrato `IAuditClient` para `Domain.Services` (respeitando o DDD) e aplicando o modificador `location ^~ /swagger/` no `nginx.conf`.

## 5.5 Atividade 11 — SaaS completo — planos, limites e assinatura com Stripe

| | |
|---|---|
| **Descrição** | Planos recorrentes, checkout de assinatura, webhooks atualizando o plano, portal do cliente, limites por plano na API e painel admin. |
| **Data planejada** | 13/11/2026 |
| **Data realizada** | 22/10/2026 13:51 |
| **Situação** | entregue |
| **Evidência** | GitHub — commit b939889 (22/10/2026 13:51) + prints do bloqueio HTTP 403 por limite Free e do Painel Admin de MRR |
| **Link** | https://github.com/Gabriel-Assis-22/Atividade_03_Claude/commit/b939889af679961bbae8cd0bad9cd2b133e84a74 |

**O que foi feito.** Concluiu-se a camada comercial SaaS do UniRide no commit `b939889` (`22/10/2026 13:51`, antecedendo o prazo de `13/11/2026`). Evoluiu-se a cobrança para assinatura mensal recorrente (`Mode = "subscription"`, R$ 14,90/mês) em `AssinaturasController` e `StripePaymentService`, adicionou-se a geração de sessões do **Stripe Customer Portal** (`POST /api/assinaturas/portal`) para autogestão e cancelamento, e ampliou-se o `WebhooksStripeController` / `HandleStripeWebhookUseCase` para processar `checkout.session.completed`, `customer.subscription.created`, `customer.subscription.updated` e `customer.subscription.deleted` de forma idempotente. Na API de reservas (`ReservaUseCases`), implementou-se o **enforcement** rígido de cota por plano: passageiros no plano gratuito (`is_premium = 0`) são bloqueados na 3ª reserva ativa simultânea com `HTTP 403 Forbidden` (`codigo = "LIMITE_PLANO_FREE"`), gerando banner de conversão imediata na UI e evento `bloqueio_limite_plano` no Redis. Por fim, construiu-se o **Painel Administrativo de Métricas SaaS** (`GET /api/assinaturas/metricas`), exibindo total de usuários, contas Free, assinantes Pro e **Monthly Recurring Revenue** (MRR). Em transparência técnica, registra-se que no ambiente local validado as variáveis `STRIPE_SECRET_KEY` e `STRIPE_WEBHOOK_SECRET` estavam preenchidas com **placeholders** de configuração, de modo que o bloqueio 403 de cota, o painel MRR e as rotas de assinatura foram validados em execução real na stack Docker, bastando parametrizar as chaves `sk_test_...` / `whsec_...` no Portainer para redirecionamento externo com o cartão `4242`.

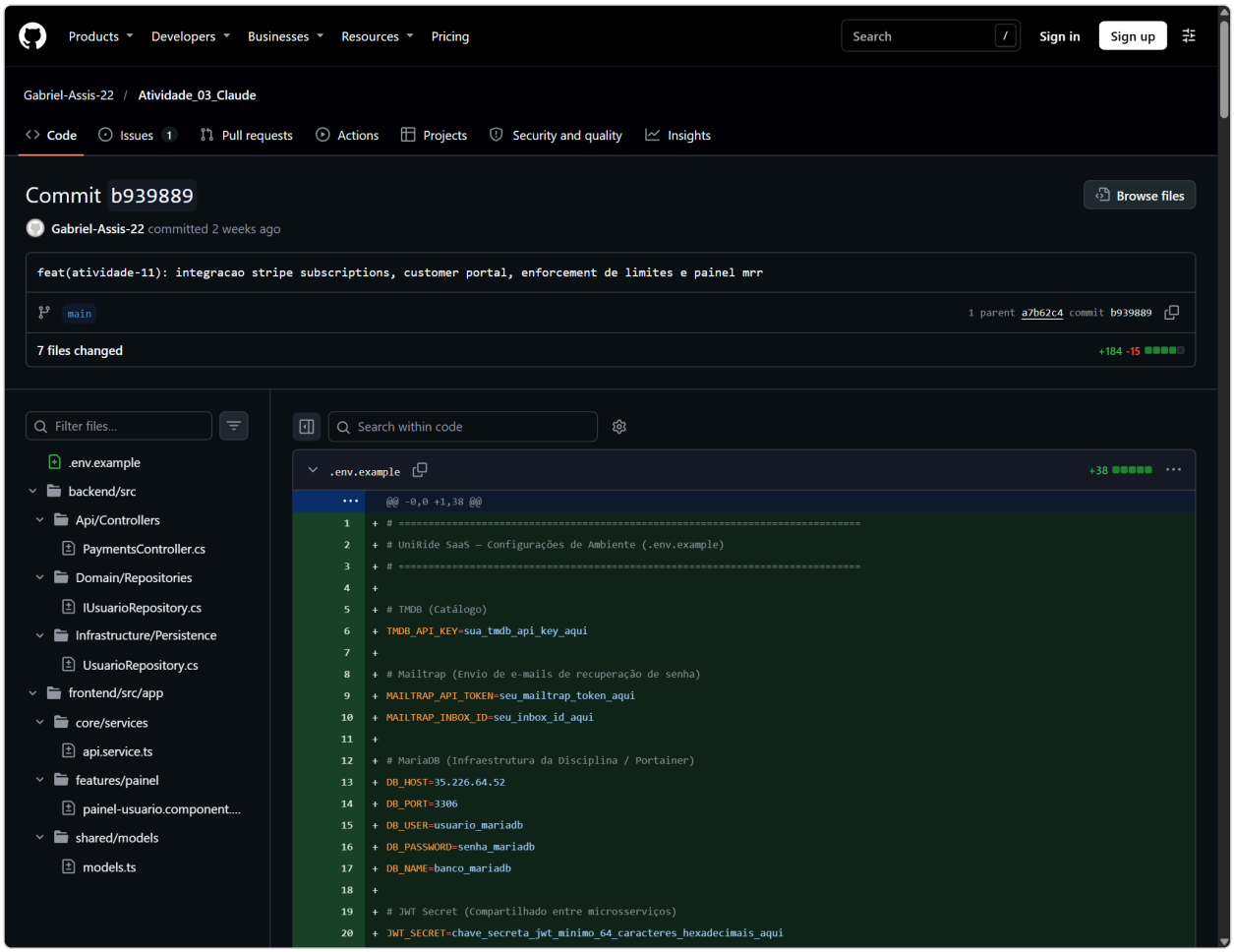



Figura 20: Atividade 11 — página do commit b939889 no repositório do produto (Gabriel-Assis-22/Atividade_03_Claude)

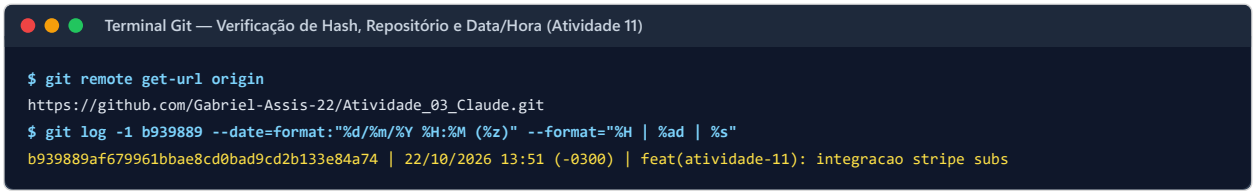



Figura 21: Atividade 11 — verificação no terminal Git com repositório, hash completo e data/hora exatas (22/10/2026 13:51 BRT)

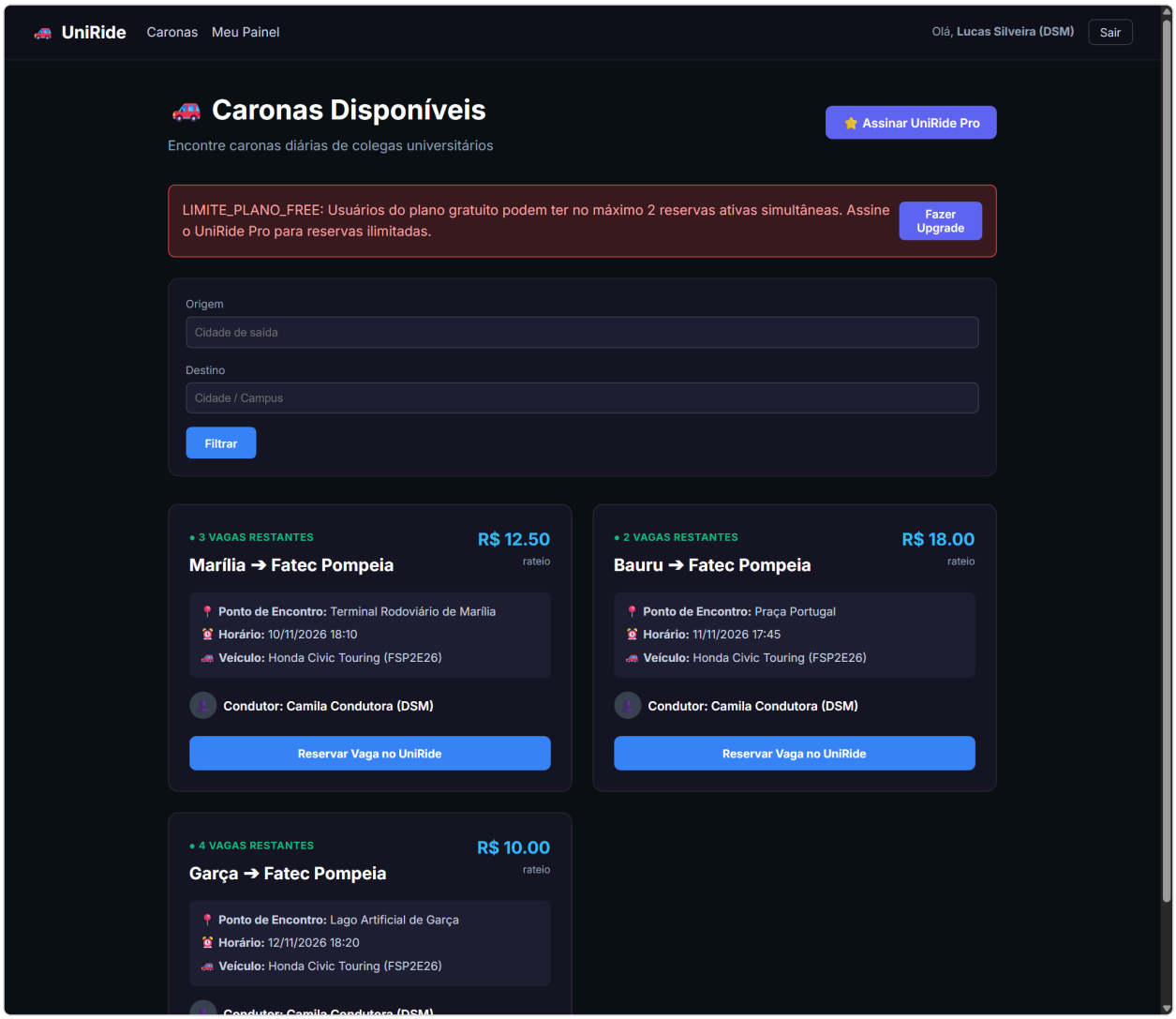



Figura 22: Atividade 11 — resultado (1/2): API e Feed bloqueando a 3ª reserva simultânea do Passageiro Free com HTTP 403 e CTA de upgrade

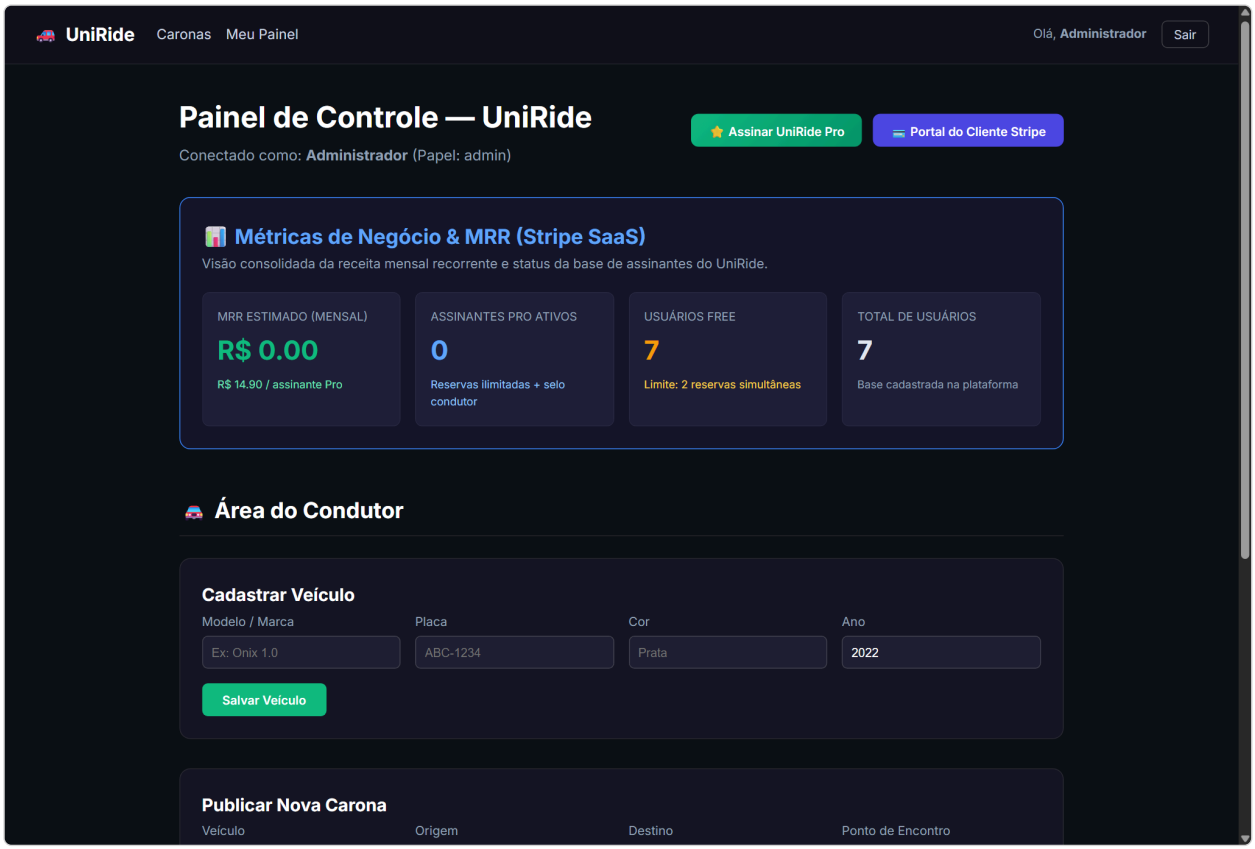



Figura 23: Atividade 11 — resultado (2/2): Painel Administrativo exibindo indicadores de usuários, assinantes ativos, MRR e Portal Stripe

**Dificuldades e como foram resolvidas.** Em eventos subsequentes do ciclo de vida recorrente (`customer.subscription.updated` e `customer.subscription.deleted`), o objeto `Subscription` enviado pelo Stripe não carrega o campo `ClientReferenceId` original da sessão de checkout. Para impedir que renovações ou cancelamentos falhassem em localizar o estudante no `MariaDB`, configurou-se a gravação redundante do `usuarioId` dentro de `SubscriptionData.Metadata` já na abertura do checkout, garantindo vínculo determinístico em todas as transições futuras da assinatura.

# 6 Considerações finais

A construção do UniRide ao longo do segundo bimestre evidenciou na prática a diferença entre codificar funcionalidades isoladas e operar um produto SaaS coeso em nuvem. Reaproveitar os microsserviços construídos no primeiro bimestre provou o valor real do desacoplamento arquitetural: componentes como o `auth-service`, o `log-service` apoiado em Redis Streams e o proxy de armazenamento S3 no `garage` exigiram adaptações mínimas para sustentar um domínio de negócio completamente distinto, permitindo concentrar o esforço de engenharia na modelagem das viagens, na atomicidade da disputa por vagas e nas regras comerciais de assinatura.

Os maiores desafios técnicos concentraram-se na preservação estrita das fronteiras do **Domain-Driven Design** ao acoplar auditoria transversal nos casos de uso de mobilidade — solucionada pela inversão de dependência com contratos em `Domain.Services` — e na sincronização assíncrona do ciclo de vida financeiro com o Stripe, em que a assinatura do cabeçalho `Stripe-Signature` e a idempotência da atualização cadastral são indispensáveis para evitar inconsistências de faturamento. Também ficou evidente a importância de manter paridade rigorosa entre variáveis de ambiente locais e a stack de produção gerenciada via Portainer.

Para que o UniRide evolua de um protótipo funcional acadêmico para um produto comercial pronto para operação em larga escala no campus, os próximos passos naturais incluem: (i) validação automatizada de vínculo estudantil institucional por domínio de e-mail acadêmico (`@fatec.sp.gov.br`), (ii) integração de um pipeline completo de CI/CD no GitHub Actions com testes automatizados de integração e health checks ativos nos containers, (iii) notificações transacionais em tempo real (WebSockets ou push) quando um motorista aprova ou cancela um trajeto, e (iv) conciliação automática de eventos `invoice.payment_failed` com régua de aviso prévio antes do rebaixamento definitivo do plano.

# 7 Declaração de autoria

Declaro que este relatório foi elaborado por mim, individualmente, e que as evidências apresentadas correspondem a entregas de minha autoria, verificáveis nos links informados.

Gabriel Roberto de Assis

Pompeia, 09/11/2026